Introducción a Python para Big Data : 🎉

Este notebook te guiará a través de los conceptos fundamentales de Python y la librería Pandas, esenciales para el análisis de datos.

1. Primeros pasos con Python 🆎

Comenzaremos con los conceptos básicos de Python, como la creación de variables y la impresión de valores.

```
# Asignación y tipos de datos
a = 1 + 1
b = "Big Data"

# Imprimir variables en pantalla
print(a)
print(b)
```

2. Manejo de datos con Pandas ⛵

Pandas es una librería fundamental para trabajar con datos tabulares. Un DataFrame de Pandas es una estructura similar a una hoja de cálculo, con filas y columnas.

Crear un DataFrame

Podemos crear un DataFrame a partir de listas de Python.

```
# Listas de datos
lista_personajes = ['Luke Skywalker', 'C-3PO', 'Darth Vader']
lista_altura = [172, 167, 202]

# Crear el DataFrame
import pandas as pd

d_star_wars = pd.DataFrame({
    'personaje': lista_personajes,
    'altura': lista_altura
})

# Mostrar el DataFrame y sus tipos de datos
print(d_star_wars)
print("---")
print(d_star_wars.dtypes)
```

3. Cargar y explorar un archivo ✈

En el mundo real, los datos rara vez se crean manualmente. Veamos como cargar un archivo CSV directamente desde una URL.

```
#Antes de comenzar tenemos que instalar la librería funpymodeling que me permite conocer información estadística de los dat
%%capture
!pip install funpymodeling
```

```
# Cargar un archivo CSV desde una URL
url = 'https://raw.githubusercontent.com/fpineyro/homework-0/master/starwars.csv'
d_star_wars_url = pd.read_csv(url)

# Explorar el DataFrame
# Muestra las primeras 5 filas
print("Primeras 5 filas:")
print(d_star_wars_url.head(7))

# Muestra el número de filas y columnas (filas, columnas)
print("\nDimensiones del DataFrame:")
print(d_star_wars_url.shape)

# Muestra un resumen estadístico de las variables
# ¡Asegúrate de instalar esta librería antes!
# !pip install funpymodeling
from funpymodeling.exploratory import status
print("\nEstadísticas del DataFrame:")
status(d_star_wars_url)
```

4. Filtrar y seleccionar datos ⛳

Para seleccionar datos específicos de tu DataFrame, usa loc. Con loc puedes seleccionar filas y columnas por sus nombres o etiquetas.

El formato loc[filas, columnas] te permite seleccionar datos usando las etiquetas de filas y columnas.

[0, :]: El primer valor (0) selecciona la fila con la etiqueta 0. Los dos puntos (:) significan todas las columnas.

[:, 'nombre_columna']: Los dos puntos (:) seleccionan todas las filas. El nombre de la columna selecciona una columna específica.

4.1. Seleccionar columnas

```
# Seleccionar una sola columna
#d_star_wars['personaje']

# Seleccionar múltiples columnas
d_star_wars[['personaje', 'altura']]
```

4.2. Seleccionar filas

Podemos seleccionar filas basándote en su posición (índice).

Nota: el uso de 0:2 en loc es inclusivo, lo que significa que incluye los índices 0, 1 y 2.

```
# Seleccionar la primera fila
d_star_wars.loc[0, :]
```

```
# Seleccionar las 3 primeras filas
d_star_wars.loc[0:2, :]
```

4.3. Filtrar con condiciones

Podemos usar una condición para filtrar filas que cumplan con un criterio específico. Por ejemplo, encontrar todos los personajes cuya altura sea mayor que 200.

```
# Crear un filtro (devuelve True/False para cada fila)
filtro_altura = d_star_wars['altura'] > 200

# Aplicar el filtro para obtener solo los registros que cumplen la condición
d_star_wars.loc[filtro_altura, :]
```

4.4. Reemplazar valores

Podemos combinar el filtrado con la asignación para cambiar valores específicos en el DataFrame.

```
# Reemplazar la altura de Darth Vader por 185
d_star_wars.loc[d_star_wars['personaje'] == 'Darth Vader', 'altura'] = 185

# Verificar el cambio
print(d_star_wars.loc[d_star_wars['personaje'] == 'Darth Vader', :])
```

5. Manejar valores nulos (NaN) ♒

5.1. Los valores faltantes (NaN) son muy comunes. Es crucial saber cómo identificarlos y manejarlos.

```
# Identificar nulos por columna
print("Número de nulos en cada columna:")
print(d_star_wars_url.isnull().sum())
```

```
d_star_wars_url
```

```
# Reemplazar valores nulos en columnas categóricas
valores_a_rellenar = {'homeworld': 'nulo_world', 'species': 'nulo_species'}

d_star_wars_url.fillna(value=valores_a_rellenar, inplace=True)
print(d_star_wars_url)
```

```
#Vemos cual es el promedio de altura
print(d_star_wars_url['height'].mean())
```

```
# Reemplazar valores nulos en columnas numéricas con el promedio
promedio_altura = d_star_wars_url['height'].mean()


d_star_wars_url['height'] = d_star_wars_url['height'].fillna(promedio_altura)
d_star_wars_url
```

```
# Verificar que los nulos fueron reemplazados
status(d_star_wars_url['height'])
```

5.2. Eliminar registros con valores nulos A veces, en lugar de reemplazar los valores nulos, lo mejor es eliminar por completo las filas o columnas que los contienen. El método dropna() de Pandas te permite hacerlo fácilmente.

Por defecto, dropna() elimina cualquier fila que contenga al menos un valor nulo. Es una forma rápida de limpiar el conjunto de datos si las filas con valores faltantes no son importantes para tu análisis.

```
# Eliminar las filas con valores nulos
d_star_wars_url.dropna(inplace=True)
status(d_star_wars_url)
```

6. Crear nuevas columnas 🏗️

Puedes crear una nueva columna en un DataFrame y asignarle valores basados en una condición. Es una forma muy poderosa de derivar información a partir de los datos existentes.

```
# Asignamos True si la altura es > 180, de lo contrario, False
d_star_wars['is_tall'] = d_star_wars['altura'] > 180

# Muestra el DataFrame con la nueva columna
print(d_star_wars)
```

7. Copiar un DataFrame ©

Cuando realizas cambios en un DataFrame y quieres conservar la versión original, es una buena práctica hacer una copia. Esto te permite manipular la nueva versión sin afectar la original.

```
#Hacer una copia del DataFrame
d_star_wars_copy = d_star_wars.copy()

# Ahora puedes trabajar con d_star_wars_copy sin afectar el original
d_star_wars_copy
```

Ejercicios 🔰

Aquí hay una serie de ejercicios para aplicar lo que aprendiste. ¡No te frustres si te toma tiempo! La práctica es la clave.

Ejercicio 1. Carga el archivo heart_disease.csv en un DataFrame llamado d_hd. URL: https://raw.githubusercontent.com/fpineyro/homework-0/master/heart_disease.csv

```
##Ejercicio 1

import pandas as pd

url = "https://raw.githubusercontent.com/fpineyro/homework-0/master/heart_disease.csv"

d_hd = pd.read_csv(url)

d_hd.head()
```

	age	gender	chest_pain	resting_blood_pressure	serum_cholestoral	fasting_blood_sugar	resting_electro	max_heart_rate
0	63	male	1	145	233	1	2	150
1	67	male	4	160	286	0	2	108
2	67	male	4	120	229	0	2	129
3	37	male	3	130	250	0	0	187
4	41	female	2	130	204	0	2	172

Próximos pasos: Generar código con d_hd New interactive sheet

Ejercicio 2. Muestra los primeros 5 y los últimos 5 registros de d_hd.

```
##Ejercicio 2
# mostrar los primeros 5 registros
d_hd.head()

# mostrar los últimos 5 registros
d_hd.tail()
```

	age	gender	chest_pain	resting_blood_pressure	serum_cholestoral	fasting_blood_sugar	resting_electro	max_heart_rat
298	45	male	1	110	264	0	0	13
299	68	male	4	144	193	1	0	14
300	57	male	4	130	131	0	0	11
301	57	female	2	130	236	0	2	17
302	38	male	3	138	175	0	0	17

Ejercicio 3. Selecciona e imprime las columnas age y has_heart_disease.

```
##Ejercicio 3

# seleccionar e imprimir solo las columnas age y has_heart_disease
print(d_hd[["age", "has_heart_disease"]])
```

```
     age has_heart_disease
0     63                no
1     67               yes
2     67               yes
3     37                no
4     41                no
..   ...               ...
298   45               yes
299   68               yes
300   57               yes
301   57               yes
302   38                no

[303 rows x 2 columns]
```

Ejercicio 4. Crea un filtro para encontrar todos los registros donde colesterol (serum_cholestoral) sea mayor que 250.

```
##Ejercicio 4

# filtrar registros donde el colesterol sea mayor a 250
filtro = d_hd[d_hd["serum_cholestoral"] > 250]

# mostrar resultados
filtro
```

	age	gender	chest_pain	resting_blood_pressure	serum_cholestoral	fasting_blood_sugar	resting_electro	max_heart_rat
1	67	male	4	160	286	0	2	10
6	62	female	4	140	268	0	2	16
7	57	female	4	120	354	0	0	16
8	63	male	4	130	254	0	2	14
11	56	female	2	140	294	0	2	15
...	...	...	...	...	...	...	...	.
280	57	male	4	110	335	0	0	14
281	47	male	3	130	253	0	0	17
285	58	male	4	114	318	0	1	14
291	55	female	2	132	342	0	0	16
298	45	male	1	110	264	0	0	13

Ejercicio 5. Usa la función status() para identificar qué variables tienen valores nulos.

```
##Ejercicio 5
# contar valores nulos por columna
d_hd.isnull().sum()
```

	0
age	0
gender	0
chest_pain	0
resting_blood_pressure	0
serum_cholestoral	0
fasting_blood_sugar	0
resting_electro	0
max_heart_rate	0
exer_angina	0
oldpeak	0
slope	0
num_vessels_flour	4
thal	2
heart_disease_severity	0
exter_angina	0
has_heart_disease	0

dtype: int64

Ejercicio 6. Reemplaza los valores nulos de la columna thal por un valor de 999.

```
##Ejercicio 6
# reemplazar valores nulos en la columna 'thal' por 999
d_hd["thal"] = d_hd["thal"].fillna(999)

# verificar que ya no haya nulos en esa columna
d_hd["thal"].isnull().sum()
```

```
np.int64(0)
```

Ejercicio 7. Filtrar por múltiples condiciones:

Utiliza el DataFrame d_hd y filtra para encontrar todos los registros de pacientes que tienen más de 60 años Y un nivel de colesterol (serum_cholestoral) superior a 250.

```
##Ejercicio 7

# filtrar pacientes con más de 60 años Y colesterol mayor a 250
filtro = d_hd[(d_hd["age"] > 60) & (d_hd["serum_cholestoral"] > 250)]
```

```
# mostrar resultados
filtro
```

	age	gender	chest_pain	resting_blood_pressure	serum_cholestoral	fasting_blood_sugar	resting_electro	max_heart_ra
1	67	male	4	160	286	0	2	
6	62	female	4	140	268	0	2	
8	63	male	4	130	254	0	2	
32	64	male	3	140	335	0	0	
42	71	female	2	160	302	0	0	
44	61	female	4	130	330	0	2	
48	65	female	3	140	417	1	2	
70	65	female	3	155	269	0	0	
71	67	male	4	125	254	1	0	
72	62	male	4	120	267	0	0	
75	65	female	3	160	360	0	2	
83	68	male	3	180	274	1	2	
90	66	male	4	120	302	0	2	
94	63	female	3	135	252	0	2	
103	71	female	3	110	265	1	2	
108	61	male	4	120	260	0	0	
110	61	female	4	145	307	0	2	
114	62	female	3	130	263	0	0	
118	63	male	4	130	330	1	2	
119	65	male	4	135	254	0	2	
121	63	female	4	150	407	0	2	
124	65	male	1	138	282	1	2	
137	62	male	2	120	281	0	2	
143	64	male	3	125	309	0	0	
152	67	female	3	115	564	0	2	
155	70	male	4	130	322	0	2	
159	68	male	3	118	277	0	0	
161	77	male	4	125	304	0	2	
170	70	male	3	160	269	0	0	
173	62	female	4	140	394	0	2	
189	69	male	3	140	254	0	2	
193	62	female	4	138	294	1	0	
195	67	male	4	100	299	0	2	
201	64	female	4	180	325	0	0	
203	64	female	3	140	313	0	0	
218	64	female	4	130	303	0	0	
224	63	female	4	108	269	0	0	
227	67	female	3	152	277	0	0	
233	74	female	2	120	269	0	2	
252	64	male	4	128	263	0	0	
276	66	female	3	146	278	0	2	

Próximos pasos: Generar código con `filtro` New interactive sheet

Ejercicio 8. Reemplazar valores en una columna:

Selecciona la columna slope del DataFrame d_hd. Reemplaza el valor 1 por el string 'ascendente' y el valor 2 por 'descendente'. Muestra el resultado (no necesitas guardarlo en el DataFrame original).

```
##Ejercicio 8
# seleccionar la columna slope y reemplazar valores
d_hd["slope"].replace({1: "ascendente", 2: "descendente"})
```

	slope
0	3
1	descendente
2	descendente
3	3
4	ascendente
...	...
298	descendente
299	descendente
300	descendente
301	descendente
302	ascendente

303 rows × 1 columns

dtype: object

Ejercicio 9. Crear una nueva columna:

En el DataFrame d_hd, crea una nueva columna llamada is_healthy. Asigna el valor True si el paciente no tiene enfermedad del corazón (es decir, el valor en heart_disease_severity es 0) y False en caso contrario. Muestra el DataFrame con la nueva columna

```
##Ejercicio 9
# crear nueva columna 'is_healthy'
# True si heart_disease_severity == 0, False en caso contrario
d_hd["is_healthy"] = d_hd["heart_disease_severity"] == 0

# mostrar el DataFrame con la nueva columna
d_hd.head()
```

sting_blood_pressure	serum_cholestoral	fasting_blood_sugar	resting_electro	max_heart_rate
145	233	1	2	150
160	286	0	2	108
120	229	0	2	129
130	250	0	0	187
130	204	0	2	172

Próximos pasos: Generar código con d_hd New interactive sheet

Ejercicio 10. Eliminar registros con nulos:

Crea una copia del DataFrame d_hd llamada d_hd_clean. Usa el método dropna() para eliminar todos los registros que contengan al menos un valor nulo. Luego, usa status() para verificar que ya no hay valores nulos en d_hd_clean.

```
##Ejercicio 10
# crear una copia del DataFrame
d_hd_clean = d_hd.copy()

# eliminar filas con al menos un valor nulo
d_hd_clean = d_hd_clean.dropna()

# verificar que ya no haya valores nulos
d_hd_clean.isnull().sum()
```

	0
age	0
gender	0
chest_pain	0
resting_blood_pressure	0
serum_cholestoral	0
fasting_blood_sugar	0
resting_electro	0
max_heart_rate	0
exer_angina	0
oldpeak	0
slope	0
num_vessels_flour	0
thal	0
heart_disease_severity	0
exter_angina	0
has_heart_disease	0
is_healthy	0

dtype: int64